

Prompt Engineering Techniques for Ensuring Correctness in Code Generation

Research Project

Automação no Desenvolvimento de Software 2025/2026 • 1º Semestre

André Cardoso
2022222265
uc2022222265@student.uc.pt

Eduardo Marques
2022231584
uc2022231584@student.dei.uc.pt

João Cardoso
2022222301
uc2022222301@student.uc.pt

RESUMO

A geração automática de código utilizando Large Language Models (LLMs) tem revolucionado o desenvolvimento de software, mas levanta questões críticas sobre a correção do código produzido. Este trabalho investiga técnicas de prompt engineering que melhoram a precisão e confiabilidade do código gerado por LLMs. São analisadas técnicas como Structured Prompting, Few-shot Prompting, Chain-of-Thought, Program-Aided Language Models, Tree of Thoughts e ReAct, demonstrando através de casos de estudo como prompts bem estruturados podem reduzir erros de sintaxe, lógica e segurança. Os resultados indicam que a adoção de prompts estruturados pode reduzir significativamente a taxa de erros lógicos quando comparado a prompts ingênuos. Conclui-se que a combinação entre prompt engineering adequado, revisão humana e testes automatizados aumenta substancialmente a confiabilidade do código gerado, embora desafios relacionados à escalabilidade e segurança se mantenham.

I. INTRODUÇÃO

A. Contextualização

A geração automática de código utilizando Large Language Models (LLMs) tem se tornado cada vez mais relevante no desenvolvimento de software. Esta tecnologia permite que programadores gerem trechos de código de forma rápida, automatizem tarefas e auxiliem na resolução de problemas complexos. No entanto, embora essa automação ofereça ganhos significativos de produtividade, são levantadas dúvidas em relação à correção do código gerado. Falhas ou erros podem causar impactos sérios, especialmente em sistemas críticos, tornando a verificação e validação do código uma necessidade fundamental.

B. Problema

Apesar do potencial presente na geração automática de código, garantir a correção do que é produzido ainda é uma grande dificuldade. Os modelos de linguagem podem gerar código semanticamente correto, mas logicamente incorreto ou

incompleto, resultando em falhas inesperadas durante a sua execução, tornando essencial o desenvolvimento de estratégias que aumentem a confiabilidade do código gerado.

C. Objetivo

O objetivo deste trabalho é analisar e comparar diferentes técnicas de prompt engineering que podem ser aplicadas para melhorar a correção do código gerado pelos modelos de linguagem. Procura-se compreender como a formulação de prompts, exemplos fornecidos e estratégias de orientação podem influenciar positivamente a precisão do código produzido, contribuindo para práticas mais confiáveis e eficientes no desenvolvimento assistido por IA.

II. FUNDAMENTAÇÃO TEÓRICA

A. Geração de Código com LLMs

Os Large Language Models (LLMs), como OpenAI's GPT-4 e ChatGPT (que usa modelos GPT), têm demonstrado uma capacidade significativa de gerar código a partir de descrições com uma linguagem natural, perceptível por seres humanos [1]. Estes modelos aprendem padrões de programação a partir de grandes volumes de código fonte disponíveis publicamente e podem gerar soluções para problemas variados em diferentes linguagens como Python, Java, entre outras. A aplicação desses modelos é ampla, incluindo geração de funções, scripts automatizados, correção de bugs e até documentação de código. Apesar do seu enorme potencial, a geração automática de código nem sempre produz resultados corretos ou eficientes, sendo vulnerável a erros de sintaxe, lógica ou segurança.

B. Arquitetura Transformer e Mecanismos de Atenção

Os LLMs modernos baseiam-se na arquitetura Transformer [2], que substituiu redes recorrentes (RNNs/LSTMs) através do mecanismo de self-attention, permitindo processamento paralelo e captura de dependências de longo alcance. Os componentes principais incluem:

- **Multi-Head Attention:** Permite ao modelo focar em diferentes "perspetivas" do contexto simultaneamente.

- **Feed-Forward Layers:** Transformações não-lineares aplicadas independentemente a cada posição.
- **Layer Normalization:** Estabiliza o treino em redes profundas.
- **Positional Encoding:** Injeta informação sobre a ordem sequencial dos tokens.

C. Problemas na Correção dos Códigos Gerados

A geração automática de código apresenta diversos desafios relacionados com a sua correção [3], incluindo:

- **Erros de sintaxe:** Impedem a execução do código, como é o caso de parênteses mal posicionados, ou até mesmo falhas na sintaxe da própria linguagem de programação em uso.
- **Erros de lógica:** Mesmo que seja executável, há situações em que o código não atende corretamente à especificação desejada.
- **Problemas de desempenho:** Ocorrem quando as soluções geradas são funcionais, mas pouco eficientes, causando impacto em sistemas de grande escala.
- **Vulnerabilidades de segurança:** A geração automática pode produzir código com exploits, como injeção de SQL ou manipulação inadequada de dados sensíveis.

Esses problemas realçam a necessidade de mecanismos que aumentem a confiabilidade do código gerado, incluindo prompt engineering adequada.

D. Conceito de Prompt Engineering e a sua Importância

Prompt Engineering é fundamental porque influencia diretamente a qualidade, a relevância e a precisão dos resultados gerados por LLMs [4]. No contexto de geração de código, um prompt bem elaborado ajuda a garantir que o modelo compreenda a intenção do usuário e produza respostas significativas, minimizando a necessidade de pós-processamento excessivo. Prompt Engineering é, portanto, uma ferramenta essencial para reduzir erros e garantir que o código gerado atenda às especificações requisitadas.

III. TÉCNICAS DE PROMPT ENGINEERING

Prompt Engineering envolve estratégias para orientar modelos de IA na produção de resultados desejados. As principais técnicas incluem:

A. Structured Prompting (6-element format)

Um prompt bem planeado deve estruturar-se como uma mini-especificação [5], alinhada com ISO/IEC/IEEE 29148:

- **Role:** Define comportamento e assunções (e.g., “És um programador sénior Python”).
- **Tarefa:** Explicita a intenção (e.g., “Cria um endpoint FastAPI...”).
- **Restrições:** Garante qualidade e conformidade (e.g., “Usa Decimal para valores monetários, não float”).
- **Exemplos e Casos de Teste:** Fornece inputs e outputs esperados para validar o comportamento (e.g., “Input: [1,2,3], Output: 6”).

- **Output:** Impõe consistência e compatibilidade com ferramentas (e.g., “Retorna apenas código em bloco Markdown”).
- **CrITÉrios de Avaliação:** Define o que é “bom” (e.g., “Código deve passar em testes unitários fornecidos”).

Exemplo de prompt:

“[ROLE] És um programador sénior Python especializado em backend.

[TASK] Escreve um endpoint FastAPI para calcular totais de faturas com descontos.

[CONSTRAINTS] Inclui validação de input. Usa decimal, não float, para valores monetários. Sem bibliotecas externas.

[EXAMPLES] Input: {valor: 100, desconto: 0.1}, Output: 90.0; Input: {valor: 50, desconto: 0.2}, Output: 40.0

[OUTPUT] Retorna apenas código com comentários e casos de teste num bloco Markdown.

[EVALUATION] O código deve passar em testes unitários que validam cálculos corretos e tratamento de erros.”

B. Few-shot Prompting

Nesta abordagem, o modelo recebe inputs e outputs esperados para que consiga aprender a desenvolver a tarefa desejada pelo utilizador [6]. Ter contexto para se basear ajuda o modelo a entender melhor o output desejado.

Exemplo de prompt:

“Input: [1, 2, 3], Output esperado: 6

Input: [4, 5, 6], Output esperado: 15

Escreva uma função em Java que funcione corretamente para os exemplos acima e para qualquer lista de números.”

Ao incluir casos de teste, é possível reduzir erros estruturais ou sintáticos e validar o comportamento do código antes de executá-lo.

C. Chain-of-Thought (CoT) / Step-by-Step

Esta técnica avançada fornece raciocínio passo a passo para orientar o modelo [7]. Dividir uma tarefa complexa em etapas menores ajuda o modelo a alcançar melhor compreensão da linguagem e a gerar resultados mais precisos.

Exemplo de prompt:

“Cria uma função em Python que ordena uma lista de strings por tamanho. Resolva o problema em etapas: primeiro define variáveis, depois escreve a lógica principal e por fim implementa a função completa.”

Este método, usado em tarefas mais complexas, aumenta a clareza e reduz a probabilidade de inconsistências no código gerado.

D. Program-Aided Language Models (PAL)

Nesta técnica, em vez de pedir ao modelo que produza apenas raciocínio em linguagem natural, pede-se ao modelo

que converta o problema num programa executável como etapa intermédia do raciocínio [8]. Os modelos de linguagem auxiliados pelo programa (PAL) diferem da abordagem Chain-of-Thought porque, em vez de gerar texto em formato livre para obter a solução, transferem a etapa de cálculo ou lógica para um tempo de execução programático, como um interpretador Python.

Desta forma, o modelo gera um programa que representa a solução do problema e a execução desse programa produz o resultado final. Este método permite reduzir erros de cálculo ou de lógica que poderiam ocorrer se o modelo tentasse resolver o problema apenas em linguagem natural, ao mesmo tempo que mantém a capacidade de decompor problemas complexos em etapas claras.

Exemplo de prompt:

“Tu és um assistente que resolve problemas gerando código Python que será executado por um interpretador para calcular a resposta.

Problema: Dada uma lista de inteiros, retorna a soma dos números ímpares multiplicada pelo número de elementos pares.

Regras:

- 1) Primeiro escreve um pequeno programa Python que lê uma lista chamada `nums`.
- 2) O código deverá calcular:
`(soma_dos_impares) * (quantidade_de_pares).`
- 3) Depois imprime apenas o resultado final como um inteiro.

Fornece apenas o código Python, sem explicações.

Exemplo de input que o interpretador dará: `nums = [1, 2, 3, 4, 5]`”

Este prompt utiliza programas como raciocínio intermédio, transferindo a execução para um interpretador, o que melhora a precisão da solução.

E. Tree of Thoughts (ToT)

A técnica Tree of Thoughts (ToT) é uma abordagem avançada de prompting engineering que visa melhorar o raciocínio estratégico e a exploração de soluções em tarefas complexas [9]. Difere da técnica Chain-of-Thought (CoT), uma vez que segue uma sequência linear de raciocínio, a ToT mantém uma árvore de pensamentos, onde cada “pensamento” é uma sequência coerente de linguagem que serve como uma etapa intermédia para resolver um problema.

Esta estrutura permite que o modelo de linguagem avalie o progresso dos pensamentos intermédios em direção à resolução do problema, utilizando um processo de raciocínio deliberado. A capacidade do modelo de gerar e avaliar pensamentos é combinada com algoritmos de procura, como pesquisa em largura (BFS) e pesquisa em profundidade (DFS), para permitir uma exploração sistemática dos pensamentos, com antecipação (*lookahead*) e retrocesso (*backtracking*).

Exemplo de prompt:

“Utilizando a técnica Tree of Thoughts. Gera possíveis “pensamentos”(expressões parciais) e

expande-os até encontrar uma expressão que resulte no número 13 usando os números [2,3,7] na ordem dada e operadores +,-,,/.*

Instruções:

- Cada pensamento é uma expressão parcial (ex.: “(2+3)”).
- Em cada passo, expande pensamentos aplicando o próximo número com cada operador.
- Mantém uma árvore limitada a profundidade 3 (usar todos os números).
- Avalia expressões e reporta a primeira que dá 13 exatamente.

Fornece pseudo-output: passos de pensamentos e expressão final.”

Este prompt instrui o modelo a gerar uma sequência de pensamentos interligados, avaliando cada um deles à medida que avança, para chegar à solução correta.

F. ReAct

A técnica ReAct combina raciocínio e ação, permitindo que os modelos de linguagem (LLMs) interajam com ferramentas externas para recuperar informações adicionais que conduzem a respostas mais confiáveis e factuais [10].

Esta abordagem permite que o modelo gere rastros de raciocínio e execute ações específicas de tarefas de forma intercalada. A geração de rastros de raciocínio permite que o modelo induza, rastreie e atualize planos de ação, além de lidar com exceções. A etapa de ação possibilita a interação e a recolha de informações de fontes externas, como bases de conhecimento ou ambientes.

Exemplo de prompt:

“Foca-te na técnica ReAct. Para cada pergunta:

- 1) Escreve o teu raciocínio breve (Thought: ...).
- 2) Se precisares de factos, faz uma ação na forma: Action: search(“termo”).
- 3) Após a ação, mostra ActionResult: “resultado simulado”.
- 4) Finalmente, dá a resposta (Answer: ...).

Usa apenas as ferramentas: `search(query)` (que retorna texto).

Base de dados local (simulada) disponível ao ambiente:

{ “Python”: “Linguagem de programação lançada em 1991 por Guido van Rossum.”, “Java”: “Linguagem de programação lançada em 1995 pela Sun Microsystems.” }

Este prompt instrui o modelo a alternar entre raciocínio e ações concretas, consultando fontes externas quando necessário para obter informações factuais.

IV. CASOS DE ESTUDO / EXEMPLOS PRÁTICOS

Para demonstrar a eficácia das técnicas de prompt engineering apresentadas, seguem dois exemplos de geração de código com prompts básicos versus prompts refinados.

A. Exemplo 1: Função de Soma de Lista (Few-shot Prompting)

Prompt Básico:

“Escreve uma função em Python que some os números de uma lista.”

Código gerado:

```
1 def soma_lista(lista):  
2     return sum(lista)
```

Problemas identificados:

- Não há tratamento de inputs inválidos.
- Nenhuma explicação sobre a lógica.
- Falta de generalização em casos não triviais.

Prompt Refinado (Few-shot Prompting):

“Input: [1, 2, 3], Output esperado: 6; Input: [4, 5, 6], Output esperado: 15
Escreve uma função em Python que produza os mesmos resultados e funcione corretamente para qualquer lista de números.”

Código gerado:

```
1 def soma_lista(lista):  
2     if not isinstance(lista, list):  
3         return 0  
4     soma = 0  
5     for num in lista:  
6         if isinstance(num, (int, float)):  
7             soma += num  
8     return soma
```

Lógica: O modelo aprendeu, a partir dos exemplos fornecidos, que deve validar os dados de inputs e somar apenas valores numéricos. Assim, generalizou a solução para outros casos semelhantes.

Melhorias obtidas:

- Redução de erros estruturais.
- Melhor compreensão da intenção do utilizador.
- Código mais robusto e validado implicitamente por exemplos.

B. Exemplo 2: Validação de Password (Chain-of-Thought)

Prompt Básico:

“Escreve uma função para validar se uma password tem pelo menos 8 caracteres.”

Código gerado:

```
1 def valida_senha(senha):  
2     return len(senha) >= 8
```

Problemas identificados:

- Apenas verifica o comprimento da senha.
- Ignora outros critérios importantes de segurança.

Prompt Refinado (Chain-of-Thought):

“Cria uma função em Python que valide uma password. Resolve o problema passo a passo:

- 1) Verifica se a password tem pelo menos 8 caracteres.
- 2) Verifica se contém letras maiúsculas e minúsculas.

3) Verifica se contém números.

4) Verifica se contém pelo menos um caractere especial.

5) Depois, implementa a função completa.”

Código gerado:

```
1 import re  
2  
3 def valida_senha(senha):  
4     if len(senha) < 8:  
5         return False  
6     if not re.search(r"[A-Z]", senha):  
7         return False  
8     if not re.search(r"[a-z]", senha):  
9         return False  
10    if not re.search(r"[0-9]", senha):  
11        return False  
12    if not re.search(r"[!@#$%^&*(),.?\":{}|<>]",  
13        senha):  
14        return False  
15    return True
```

Lógica: O modelo seguiu o raciocínio passo a passo proposto no prompt, implementando a função de forma sequencial. Esse processo reduz a probabilidade de erros lógicos e aumenta a clareza do código.

Melhorias obtidas:

- Explicação da lógica de raciocínio.
- Código mais legível e fácil de manter.
- Menor probabilidade de omitir condições de validação.

C. Exemplo 3: Jogo dos 24 (Tree of Thoughts)

Prompt Básico:

“Usando os números 4, 9, 10, 13, cria expressões matemáticas que resultem em 24.”

Código gerado:

```
1 def resolver_24(numeros):  
2     return f"({numeros[0]} + {numeros[1]}) * ({numeros[2]} - {numeros[3]})"
```

Problemas identificados:

- Não explora as múltiplas possibilidades de combinação.
- Não valida se a expressão realmente resulta em 24.
- Falha em casos onde a primeira tentativa não funciona.

Prompt Refinado (Tree of Thoughts):

“Tarefa: Usando os números 4, 9, 10, 13 e as operações +, -, *, /, encontra uma expressão que resulte em 24.

Segue esta estratégia:

- 1) Gera 3 possíveis passos iniciais (pensamentos) combinando dois números.
- 2) Para cada pensamento, avalia se está próximo de 24 ou se existe um caminho promissor.
- 3) Expande os pensamentos mais promissores.
- 4) Se um caminho falhar, retrocede e explora alternativas.

Exemplo de raciocínio:

- Pensamento 1: $(10 - 4) = 6$, depois $6 * (13 - 9) = 24$ ✓

- *Pensamento 2: $(13 - 9) = 4$, depois $4 * 10 = 40$, depois $40 - 4 = 36 \times$*
- *Pensamento 3: $(9 + 10) = 19$, não promissor para chegar a 24*

Implementa uma função que explore sistematicamente estas possibilidades.”

Código gerado:

```
1 from itertools import permutations, product
2
3 def avaliar_pensamento(expr, objetivo=24):
4     """Avalia o quão promissor é um pensamento
5     intermedio"""
6     try:
7         resultado = eval(expr)
8         distancia = abs(resultado - objetivo)
9         return resultado, distancia
10    except:
11        return None, float('inf')
12
13 def resolver_24_tot(numeros):
14     ops = ['+', '-', '*', '/']
15     melhor_solucao = None
16     melhor_distancia = float('inf')
17
18     # Arvore de pensamentos: explora combinacoes
19     for perm in permutations(numeros):
20         for ops_comb in product(ops, repeat=3):
21             # Nivel 1: Primeiro pensamento (combinar
22             # 2 numeros)
23             expr1 = f"({perm[0]} {ops_comb[0]} {perm
24             [1]})"
25             resultado1, dist1 = avaliar_pensamento(
26             expr1)
27
28             if resultado1 is None:
29                 continue
30
31             # Nivel 2: Expandir pensamento promissor
32             expr2 = f"({expr1} {ops_comb[1]} {perm
33             [2]})"
34             resultado2, dist2 = avaliar_pensamento(
35             expr2)
36
37             if resultado2 is None:
38                 continue
39
40             # Nivel 3: Solucao final
41             expr_final = f"({expr2} {ops_comb[2]} {
42             perm[3]})"
43             resultado_final, dist_final =
44             avaliar_pensamento(expr_final)
45
46             # Avalia e guarda o melhor caminho
47             if resultado_final == 24:
48                 return expr_final, "Solucao
49                 encontrada!"
50
51             if dist_final < melhor_distancia:
52                 melhor_distancia = dist_final
53                 melhor_solucao = expr_final
54
55     return melhor_solucao, f"Melhor aproximacao: {
56     eval(melhor_solucao)}"
57
58 # Teste
59 numeros = [4, 9, 10, 13]
60 solucao, status = resolver_24_tot(numeros)
61 print(f"Expressao: {solucao}")
62 print(f"Status: {status}")
63 print(f"Resultado: {eval(solucao)}")
```

Lógica: O modelo implementa uma árvore de pensamentos estruturada em 3 níveis:

- 1) **Geração de pensamentos:** Para cada permutação de números e operações, cria pensamentos intermédios.
- 2) **Avaliação:** Cada pensamento é avaliado pela sua distância ao objetivo (24).
- 3) **Exploração sistemática:** Utiliza pesquisa em largura implícita ao explorar todas as combinações.
- 4) **Antecipação e retrocesso:** Se um caminho não é promissor (distância grande), o algoritmo retrocede e explora outras alternativas.

A estrutura permite que o modelo mantenha múltiplos caminhos de raciocínio simultâneos, avalie a qualidade de soluções parciais, descarte caminhos não promissores e encontre a solução ótima através de exploração sistemática.

Melhorias obtidas:

- **Exploração estruturada:** Pesquisa sistemática em vez de tentativa única.
- **Avaliação de qualidade:** Cada pensamento intermédio é avaliado quanto ao seu potencial.
- **Robustez:** Encontra soluções mesmo em casos complexos através de *backtracking*.
- **Raciocínio explícito:** O processo de resolução é transparente e verificável.
- **Escalabilidade:** A abordagem funciona para problemas mais complexos com mais números ou objetivos diferentes.

D. Análise dos Casos de Estudo

Impacto das técnicas de prompt engineering:

- Maior precisão e confiabilidade dos resultados.
- Geração de código mais robusto e explicativo.
- Melhor alinhamento entre intenção do utilizador e comportamento do modelo.
- Redução de erros lógicos e sintáticos.

Observação: Mesmo com prompts refinados, recomenda-se revisão humana e testes automatizados para garantir total confiabilidade do código antes de o colocar em produção.

V. DESAFIOS E LIMITAÇÕES

Embora Prompt Engineering seja uma ferramenta poderosa para aumentar a correção do código gerado por LLMs, apresenta desafios e limitações a ter em conta.

A. Dependência da Qualidade do Prompt

A eficácia das técnicas de prompt engineering está diretamente ligada à clareza e ao detalhe presentes no prompt. Prompts mal formulados podem gerar resultados incorretos ou incompletos. Criar prompts eficientes requer experiência e conhecimento do domínio do problema.

B. Escalabilidade e Automação

Para sistemas de grande escala, a revisão manual de prompts e códigos gerados é necessária. A automação completa depende da integração com testes automatizados, verificações formais e pipelines de validação, o que aumenta a complexidade do processo.

C. Limitações Éticas e de Segurança

Modelos podem gerar código que inadvertidamente viola licenças de software ou práticas éticas de desenvolvimento. Há risco de inclusão de vulnerabilidades de segurança, mesmo com prompts cuidadosamente formuladas [11].

Em síntese, embora o Prompt Engineering contribua para a redução de erros, este não substitui a revisão humana, os testes automatizados nem as boas práticas de engenharia de software.

D. Enviesamento e Qualidade dos Dados de Treino

A qualidade e composição dos dados de treino definem fundamentalmente o que os LLMs “conhecem” e como raciocinam sobre os problemas de programação. Esta limitação estrutural não pode ser completamente mitigada através de prompt engineering.

Problemas identificados:

- **Dados desequilibrados:** Os modelos apresentam forte enviesamento para linguagens populares (Python, JavaScript, C) em detrimento de linguagens especializadas (Rust, Ada, Haskell).
- **Código de baixa qualidade:** Repositórios públicos contêm numerosos exemplos inseguros, ineficientes ou desatualizados que são incorporados no modelo durante o treino.
- **Inconsistências temporais:** Modelos treinados com código antigo podem gerar padrões obsoletos, APIs descontinuadas ou práticas que já não são recomendadas.
- **Ausência de contexto de segurança:** Muitos exemplos de treino não incluem validação, tratamento de erros ou considerações de segurança adequadas.

Implicações práticas:

- A competência do modelo varia drasticamente pelo domínio técnico abordado no prompt, linguagem e tipo da tarefa.
- As recomendações podem ser inconsistentes mesmo para prompts estruturalmente semelhantes.
- O ajuste específico do domínio é frequentemente necessário para aplicações críticas.
- A confiança nos resultados deve ser calibrada de acordo com o domínio específico.

Estes são enviesamentos estruturais profundamente incorporados nos modelos durante o treino. Não podem ser completamente eliminados através de prompt engineering ou filtros de pós-processamento. A mitigação eficaz requer:

- Curadoria rigorosa dos conjuntos de dados de treino.
- Avaliação contínua do desempenho do modelo em diferentes domínios.
- Ajuste específico para aplicações críticas.
- Validação humana especializada em áreas de baixa cobertura dos dados de treino.

Em contextos de software crítico, é fundamental reconhecer que os LLMs possuem “pontos cegos” resultantes da qualidade desigual dos dados de treino, e que estes pontos cegos representam riscos significativos de falhas.

VI. CONCLUSÃO

Prompt Engineering mostra-se essencial para melhorar a precisão, clareza e segurança do código gerado por modelos de linguagem (LLMs). Este trabalho abordou:

- Os problemas mais comuns na geração automática de código, incluindo erros de sintaxe, falhas lógicas, baixa eficiência e vulnerabilidades de segurança.
- Conceitos fundamentais de Prompt Engineering e sua aplicação prática para aumentar a confiabilidade do código gerado.
- Técnicas de Prompt Engineering, como instruções claras, exemplos concretos, decomposição de tarefas, auto-verificação de erros e imposição de restrições de segurança.
- Estudos de caso que demonstram melhorias significativas no código quando prompts são cuidadosamente elaborados.

Apesar das limitações inerentes à geração automática, a combinação de prompts bem estruturados, revisão humana e testes automatizados pode aumentar substancialmente a qualidade e confiabilidade do código produzido. Trabalhos futuros podem explorar a integração de Prompt Engineering com verificação formal, frameworks de validação de código e sistemas de testes automatizados, ampliando ainda mais a eficácia dessas técnicas e consolidando seu papel na engenharia de software assistida por IA.

REFERÊNCIAS

- [1] G. Porto, “LLMs e GPT-4: Visão Geral,” Medium, 2023. [Online]. Available: <https://medium.com/>
- [2] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention is All You Need,” in Advances in Neural Information Processing Systems (NeurIPS), 2017, pp. 5998–6008. <https://arxiv.org/abs/1706.03762>
- [3] Vinicius, “Desenvolvimento de Software Assistido por IA,” Dev.to, 2023. [Online]. Available: <https://dev.to>
- [4] “LLMs for Code Generation,” Prompting Guide, 2025. [Online]. Available: <https://www.promptingguide.ai>
- [5] J. Robert, I. Ozkaya, and D. C. Schmidt, “Transforming Software Engineering and Acquisition with Large Language Models,” Software Engineering Institute, Carnegie Mellon University
- [6] “Few-shot Prompting,” Prompting Guide, 2025. [Online]. Available: <https://www.promptingguide.ai/pt/techniques/fewshot>
- [7] “Chain-of-Thought Prompting,” Prompting Guide, 2025. [Online]. Available: <https://www.promptingguide.ai/pt/techniques/cot>
- [8] “Program-Aided Language Models,” Prompting Guide, 2025. [Online]. Available: <https://www.promptingguide.ai/pt/techniques/pal>
- [9] “Tree of Thoughts,” Prompting Guide, 2025. [Online]. Available: <https://www.promptingguide.ai/pt/techniques/tot>
- [10] “ReAct: Reasoning and Acting,” Prompting Guide, 2025. [Online]. Available: <https://www.promptingguide.ai/pt/techniques/react>
- [11] “Prompting Techniques for Secure Code Generation,” ACM Digital Library, 2025.

Nota: Para além das referências acima listadas, os slides das aulas e as fichas práticas também foram consultados contribuindo para o desenvolvimento deste trabalho.